00 01 02 03 **04** 05 06 07 08 09 0A 0B 0C 0D

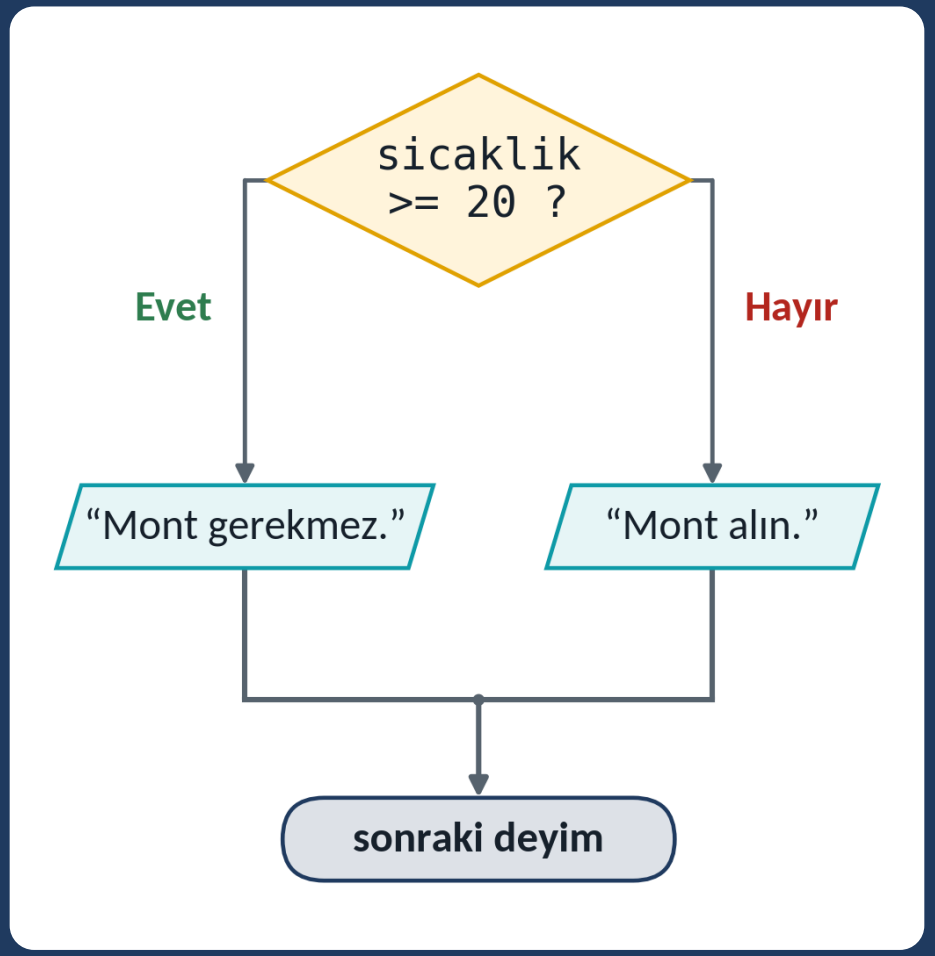
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 05

Karar Yapıları: Programın Yol Ayrımları

if, else-if, iç içe if, switch ve ?: · De Morgan · girdi doğrulama · sınır değer testi · kararların makine dilindeki karşılığı

MODÜL III · KONTROL AKIŞI



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** **Karar Yapıları**	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** Karakter Dizileri	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GEÇEN HAFTADAN

Geçen haftadan · S1 ve S2: ekranda ne görülür?

H05_K01_geri_cagirma.c · satır 10–13

```c
/* S1: tamsayı bölmesi, kalan ve cast */
int a = 7;
int b = 2;
printf("%d %d %.1f\n", a / b, a % b, (double)a / b);
```

A. 3 1 3.5

B. 3.5 1 3.5

C. 3 1 3.0

D. 4 1 3.5

H05_K01_geri_cagirma.c · satır 15–17

```c
/* S2: ilişkisel ve mantıksal ifadeler 0 ya da 1 üretir */
int x = 5;
printf("%d %d %d\n", x > 3, x > 3 && x > 8, !x);
```

A. 1 0 0

B. 1 1 0

C. 5 0 1

D. true false false

Geçen haftadan · S3 ve yanıtlar

S3: `yil = 1900` için `(yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0` kaçtır? (satır 20–22)

S1 · 3 1 3.5

`7 / 2` tamsayı bölmesi (3), `7 % 2` kalan (1), `(double)7 / 2` kayan noktalı bölme (3.5).

S2 · 1 0 0

İlişkisel/mantıksal operatörler **int 0 ya da 1** üretir; `!5` = 0 çünkü 5 "doğru"dur.

S3 · 1900 → 0

4'e ve 100'e bölünür, 400'e bölünmez: `(1 && 0) || 0` = 0. Bu hafta aynı kuralı **if zinciriyle** de yazacağız (Örnek 5).

BÜYÜK SORU

Haftanın Büyük Sorusu

Tek bir fazladan satır ve eksik süslü parantezler, milyonlarca cihazın güvenli bağlantısını nasıl devre dışı bırakabilir?

hata_avcisi/H05_HA4_goto_fail_yeniden.c · satır 14–20

```
14  if ((hata = ozet_hatasi) != 0)
15      goto fail;
16      goto fail;              /* yinelenen satır: HER durumda atlar */
17
18  printf("2. adım: sunucunun imzası doğrulanıyor\n");
19  if (imza_gecerli == 0) {
20      hata = -1;
```

GERÇEK ÇIKTI · SAHTE İMZA

```
1. adım: el sıkışma verisinin özeti
hesaplanıyor
Sonuç: doğrulama BAŞARILI, bağlantı
güvenli sayıldı
```

"2. adım" hiç yazılmadı: imza **hiç denetlenmedi**, yine de sonuç BAŞARILI.

İpucu: 16. satır girintiyle `if`'e aitmiş gibi görünüyor. **Derleyici için girinti anlamsızdır.** Neden?

GERÇEK DÜNYADAN

CVE-2014-1266 · Şubat 2014

“goto fail”: iki satırlık felaket

- Şubat 2014: Apple’ın **SecureTransport** kitaplığında, sunucunun anahtar değişimi iletisindeki imzayı doğrulayan işlevde (`SSLVerifySignedServerKeyExchange`) `goto fail;` satırı **iki kez** yazılmıştı.
- Önceki `if` süslü parantezsizdi: yalnızca ilk satırı koruyordu. İkinci satır **her durumda** çalıştı ve **son imza doğrulama adımı erişilemez** oldu.
- Hata değişkeni o anda **0 (başarı)** olduğu için doğrulama başarılı sayıldı: ortadaki adam saldırısına kapı açıldı. Düzeltme: iOS 7.0.6, iOS 6.1.6, OS X 10.9.2 ve Apple TV 6.0.2.

Kaynaklar: NIST (2014), CVE-2014-1266; Langley (2014), ImperialViolet

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 5.1	if, if-else, else-if merdiveni, iç içe if ve switch yapılarını problem türüne göre seçer ve uygular.
ÖÇ 5.2	Bileşik koşulları mantıksal operatörler ve De Morgan kurallarıyla kurar ve sadeleştirir.
ÖÇ 5.3	Tüm dalları kapsayan test durumlarını sınır değer çözümlemesi ile tasarlar.
ÖÇ 5.4	Karar yapılarının makine düzeyindeki karşılığını (karşılaştırma, koşullu atlama, atlama tablosu) açıklar.

01

if, if-else ve else-if

Koşul bir ifadedir: 0 yanlış, sıfır dışı doğru

C’de doğruluk: 0 yanlış, sıfırdan farklı her değer doğru

3	-7	0	10 > 3	10 < 3
doğru	doğru	yanlış	1	0

H05_K02_dogruluk_degeri.c · satır 26–32

```c
26  int sonuc = (10 > 3);
27  printf("(10 > 3) = %d, (10 < 3) = %d\n", sonuc, 10 < 3);
28
29  bool gecti = true;
30  bool devamsiz = false;
31  printf("gecti = %d, devamsiz = %d, sizeof(bool) = %zu\n",
32         gecti, devamsiz, sizeof(bool));
```

GERÇEK ÇIKTI (SON İKİ SATIR)

```
(10 > 3) = 1, (10 < 3) = 0
gecti = 1, devamsiz = 0, sizeof(bool) = 1
```

- `if (sayac)` ≡ `if (sayac != 0)`
- `<stdbool.h>`: `bool`, `true` (1), `false` (0)

Tek dallı if: koşul yanlışsa gövde atlanır



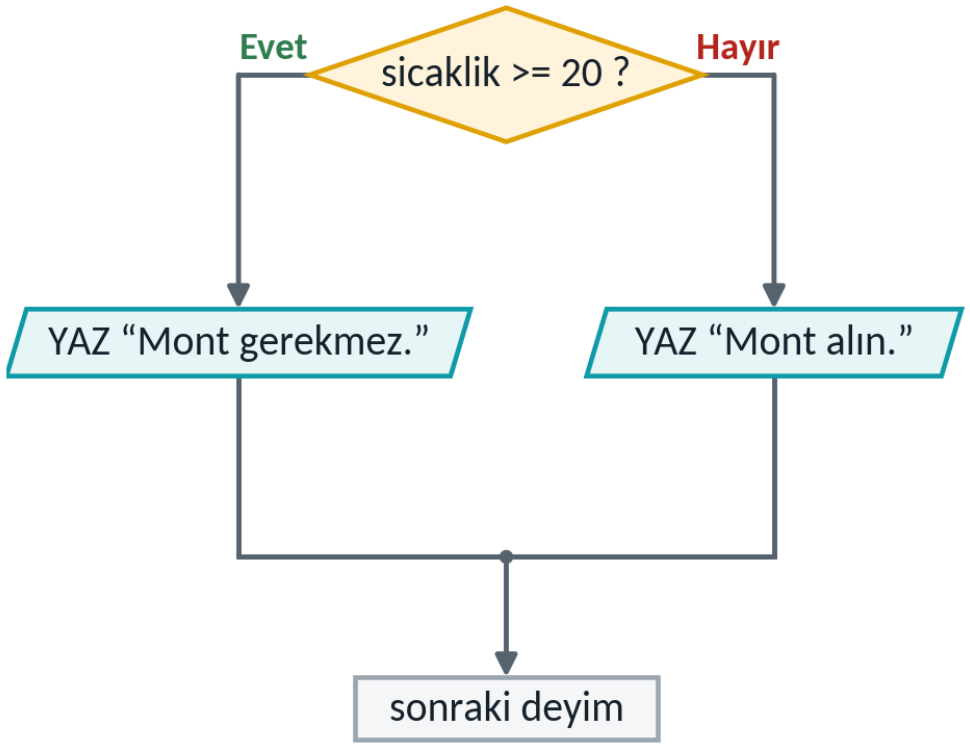
H05_K03_if_blok.c · satır 13–15

```
13      if (sicaklik > 30) {
14          printf("Sıcak bir gün: bol su için.\n");
15      }
```

- Koşul doğruysa (≠ 0) gövde çalışır.
- Koşul yanlışsa (0) gövde atlanır;
- akış if'ten sonraki deyimle sürer.
- Gövde tek satır olsa bile { } kullanın.

Akış diyagramı ve H05_K03_if_blok.c (satır 13–15)

if-else: iki yoldan tam olarak biri



H05_K03_if_blok.c · satır 18–22

```
18      if (sicaklik >= 20) {
19          printf("Mont gerekmez.\n");
20      } else {
21          printf("Mont alın.\n");
22      }
```

İki yoldan TAM OLARAK biri çalışır.

else'in kendi koşulu yoktur:

“aksi hâlde” demektir.

Programın çıktısı: “Mont gerekmez.”

H2’deki karar sembolünün C karşılığı: Evet dalı if gövdesi, Hayır dalı else gövdesi (H05_K03_if_blok.c, satır 18–22)

Blok { } bir kapsam oluşturur

H05_K03_if_blok.c · satır 24–28

```c
/* Blok bir kapsam (scope) oluşturur */
if (sicaklik > 30) {
    int fark = sicaklik - 30;        /* yalnızca bu blokta yaşar */
    printf("Eşik %d derece aşıldı.\n", fark);
}
```

- Blokta bildirilen `fark` **yalnızca o blokta yaşar**.
- Bloktan sonra da gerekecekse **bloktan önce** bildirin.
- Kapsam ve ömür: H9.

hata_avcisi/H05_HA9_blok_kapsami.c · satır 11–15

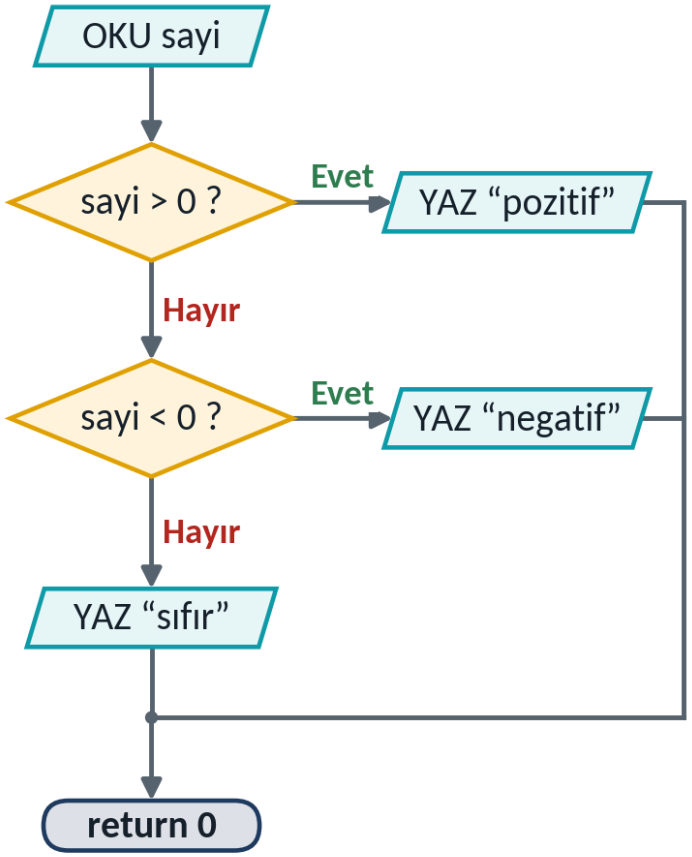
```c
if (sicaklik > 30) {
    int fark = sicaklik - 30;      /* bloğun içinde doğar */
    printf("Eşik aşıldı.\n");
}
printf("Fark: %d\n", fark);        /* blok bitti, fark yok! */
```

GCC 13 · HA9 (DERLENMEZ)

```
15:26: error: 'fark' undeclared
(first use in this function)
```

Düzeltilmiş hâli: `int fark = 0;` if'ten önce (HA9 düzeltilmiş).

else-if merdiveni: ilk doğru koşul kazanır



H05_01_isaret.c · satır 15–21

```
15      if (sayi > 0) {
16          printf("%d pozitiftir.\n", sayi);
17      } else if (sayi < 0) {
18          printf("%d negatiftir.\n", sayi);
19      } else {
20          printf("Sayı sıfırdır.\n");
21      }
```

Koşullar yukarıdan aşağıya sınanır;

ilk doğru koşulun gövdesi çalışır,

gerisine hiç bakılmaz.

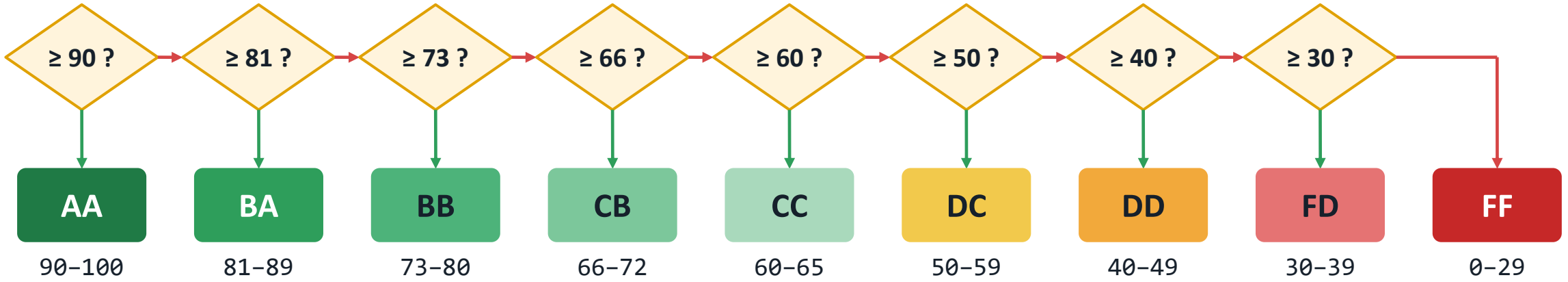
Son else: “hiçbiri doğru değilse”.

Örnek 1 (H05_01_isaret.c): koşullar yukarıdan aşağıya sınanır

Harf notu: RTEÜ tablosu, tek merdiven

Harf	AA	BA	BB	CB	CC	DC	DD	FD	FF
Puan	90–100	81–89	73–80	66–72	60–65	50–59	40–49	30–39	0–29
Katsayı	4,0	3,5	3,0	2,5	2,0	1,5	1,0	0,5	0,0
Durum	başarılı	başarılı	başarılı	başarılı	başarılı	şartlı*	başarısız	başarısız	başarısız

↓ **Evet:** harf notunu yaz, merdivenden çık → **Hayır:** bir sonraki basamağa geç



** DC: o yarıyılın YANO'su ≥ 2,00 ise başarılı. Ara sınav %40, yarıyıl sonu %60; yarıyıl sonu sınavından en az 50 gerekir, aksi hâlde FF.*
Kaynak: RTEÜ Ön Lisans ve Lisans Eğitim-Öğretim ve Sınav Yönetmeliği (2025), Madde 30. H2'deki beş kademeli tablo sadeleştirilmiş bir örnekti; artık kurumun gerçek tablosu. Ortalamanın yuvarlanması ders içi modelleme varsayımıdır.

KARŞILAŞTIR

Aralıkların sırası önemlidir

✗ Yanlış sıra
Alıştırma 5.3
95 → şartlı başarılı

alistirmalar/H05_A03_yanlis_sira.c · satır 14–20

```c
14  if (puan >= 50) {
15      printf("şartlı başarılı\n");
16  } else if (puan >= 60) {
17      printf("başarılı\n");
18  } else {
19      printf("başarısız\n");
20  }
```

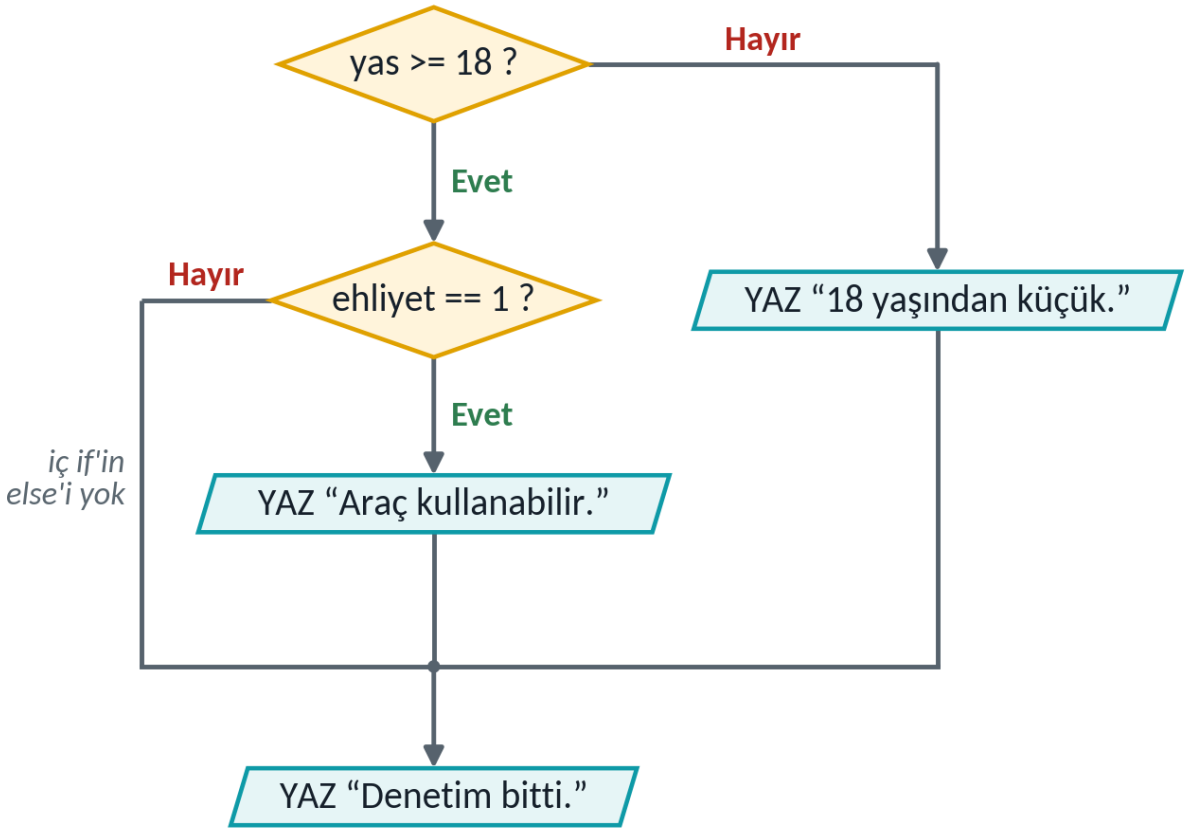
✓ Büyükten küçüğe
Örnek 13
(durum satırı)

H05_13_harf_notu_test.c · satır 36–42

```c
36  if (puan >= 60) {           /* CC ve üzeri: başarılı */
37      printf(" başarılı\n");
38  } else if (puan >= 50) {    /* DC: YANO >= 2,00 ise başarılı */
39      printf(" şartlı başarılı\n");
40  } else {
41      printf(" başarısız\n");
42  }
```

Üstteki program derlenir ve uyarı vermez; gerçek çıktı: **95 → şartlı başarılı**. “başarılı” dalına hiç ulaşılamaz (dal kapsamı!).

İç içe if: soru yalnızca ilk yanıt “evet” ise sorulur



H05_HA5_…_duzeltilmis.c · satır 12–20

```
12      if (yas >= 18) {
13          if (ehliyet == 1) {
14              printf("Araç kullanabilir.\n");
15          }
16      } else {
17          printf("18 yaşından küçük.\n");
18      }
19
20      printf("Denetim bitti.\n");
```

Her if kendi { } bloğuyla: else'in hangi if'e ait olduğu tartışmasızdır.

TUZAK

Sallanan else: girinti yalan söyleyebilir

H05_HA5_sallanan_else.c · satır 12–16 (yas = 16, ehliyet = 0)

```
12  if (yas >= 18)
13      if (ehliyet == 1)
14          printf("Araç kullanabilir.\n");
15  else                                  /* girinti yalan söylüyor */
16      printf("18 yaşından küçük.\n");
```

GCC 13 (-Wall):

```
warning: suggest explicit braces to avoid
ambiguous 'else' [-Wdangling-else]
```



else, en yakın ve else'i olmayan if'e aittir · gerçek GCC 13 uyarısı ve çalıştırma

02

Bileşik koşullar, ?: ve switch

&&, ||, De Morgan, kısa devre · case, break, bilinçli düşüş · girdi doğrulama

Aralık içinde mi, dışında mı? && ve ||

H05_K04_mantiksal_kosullar.c · satır 12–15

```c
/* Aralığın İÇİNDE mi? İki koşul birlikte: && */
if (puan >= 0 && puan <= 100) {
    printf("%d geçerli bir puan.\n", puan);
}
```

H05_K04_mantiksal_kosullar.c · satır 17–20

```c
/* Aralığın DIŞINDA mı? Biri yeter: || */
if (puan < 0 || puan > 100) {
    printf("%d geçersiz bir puan.\n", puan);
}
```

GERÇEK ÇIKTI (PUAN = 105)

```
105 geçersiz bir puan.
ic = 0, dis1 = 1, dis2 = 1
Ortalama hesaplanmadı (adet = 0).
```

VE (&&): iki koşul da doğru olmalı → aralığın **içi**

VEYA (||): biri doğruysa yeter → aralığın **dışı**

Matematikteki `0 ≤ puan ≤ 100` C'de **iki karşılaştırmanın VE'sidir**. `0 <= puan <= 100` yazmak geçerli C'dir ama başka bir şey demektir (HA6).

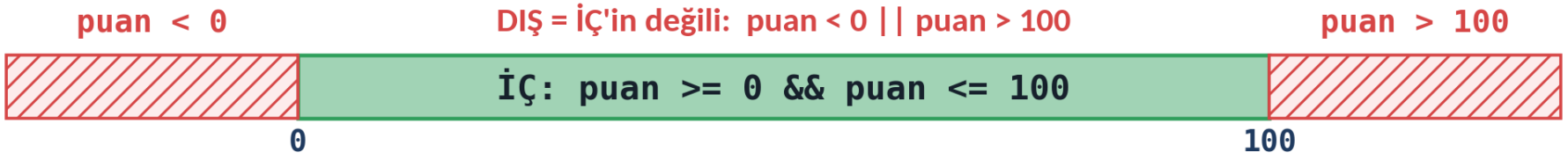
Doğruluk tabloları ve De Morgan kuralları

girdiler		VE · VEYA		De Morgan 1: eşit sütunlar		De Morgan 2: eşit sütunlar	
A	B	A && B	A || B	!(A && B)	!A || !B	!(A || B)	!A && !B
0	0	0	0	1	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	0

Değerler H05_K05_de_morgan_tablosu.c programının gerçek çıktısıdır: C'de mantıksal işlemlerin sonucu int 0 ya da 1'dir.

De Morgan: !(A && B) ≡ !A || !B ve !(A || B) ≡ !A && !B (dört satırın hepsinde eşit)

De Morgan ile olumsuz koşulu sadeleştirmek




```
!(puan >= 0 && puan <= 100)
```
başlangıç: "aralığın içinde DEĞİL"

```
!(puan >= 0) || !(puan <= 100)
```
1) && ↔ || ve her koşul değilini alır

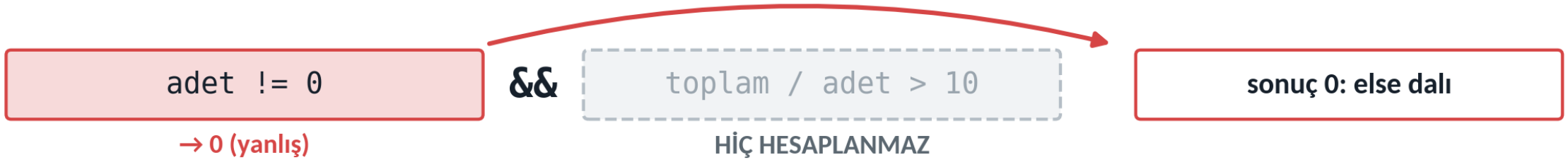
```
puan < 0 || puan > 100
```
2) karşılaştırmaların değili: >= → < · <= → >

Değil alırken: `<` ↔ `>=` · `>` ↔ `<=` · `==` ↔ `!=` · `&&` ↔ `||`

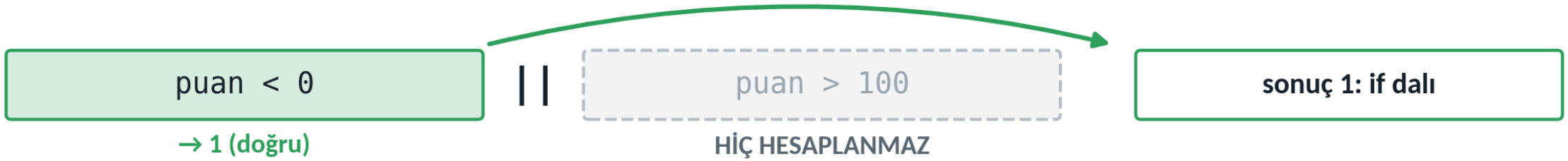
&& ↔ || yer değiştirir ve her karşılaştırma tersine döner (>= → <, <= → >, == → !=) · gerçek çıktı: ic = 0, dis1 = 1, dis2 = 1

Kısa devre: sağ taraf hiç hesaplanmayabilir

&& : sol yanlışsa sonuç kesin 0 (H05_K04_mantiksal_kosullar.c, adet = 0)



|| : sol doğruysa sonuç kesin 1 (puan = -5 için aralık dışı denetimi)



Kazanç: sıfıra bölme gibi hatalar önlenir. Kural: "güvenlik" koşulunu (adet != 0) HER ZAMAN sola yazın.

TAHMİN ET

Bu program ne yazdırır?

quiz/H05_Q04_kisa_devre.c · satır 8–14

```c
int a = 0;
int b = 10;
if (a != 0 && b / a > 2) {
    printf("X\n");
} else {
    printf("Y\n");
}
```

A X

B Y

C Çalışma zamanında sıfıra bölme hatası

D Derlenmez

Koşul operatörü ?: bir değer seçer

H05_K06_kosul_operatoru.c · satır 12–19

```c
/* koşul ? doğruysa_değer : yanlışsa_değer */
int mutlak = (n < 0) ? -n : n;
printf("|%d| = %d\n", n, mutlak);

printf("%d bir %s sayıdır.\n", n, (n % 2 == 0) ? "çift" : "tek");

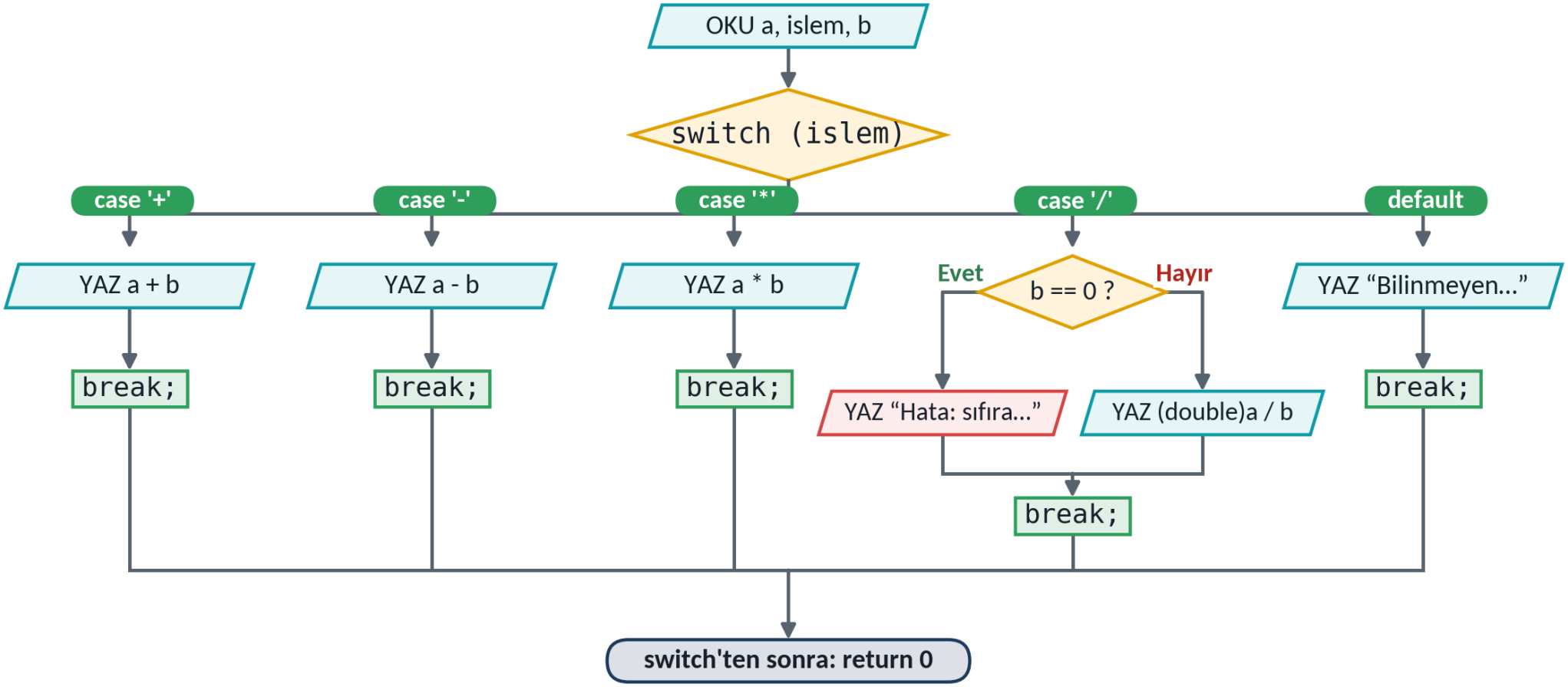
int final_notu = 45;
printf("Final barajı: %s\n", (final_notu >= 50) ? "geçildi" : "FF");
```

ÇIKTI

```
|-7| = 7
-7 bir tek sayıdır.
Final barajı: FF
```

- `koşul ? değer1 : değer2`: C'nin tek üç işlenenli operatörü
- Bir **ifadedir**: atamada, printf argümanında kullanılır
- `%s` tırnak içindeki metni yazar (ayrıntı: H13)
- Farklı **işler** yapılacaksa `if-else` daha okunaklıdır

switch: eşleşen case etiketine atla



break, switch bloğunun sonuna atlar. Case etiketleri yalnızca GİRİŞ noktalarıdır; break yoksa akış bir alttaki case'e düşer.

Case etiketleri giriş noktalarıdır: bilinçli düşüş



/* fall through */ yorumu düşüşün BİLİNÇLİ olduğunu söyler: GCC'nin -Wimplicit-fallthrough uyarısı (-Wextra) susar.

TAHMİN ET

n = 2 iken ne yazdırılır?

quiz/H05_Q03_bilincli_dusus.c · satır 8–22

```c
int n = 2;
switch (n) {
case 1:
    printf("bir ");
    /* fall through */
case 2:
    printf("iki ");
    /* fall through */
case 3:
    printf("üç ");
    break;
default:
    printf("çok ");
    break;
}
```

A iki

B iki üç

C bir iki üç

D iki üç çok

Girdi doğrulama: önce denetle, sonra kullan

H05_K09_girdi_dogrulama.c · satır 12–21

```c
printf("Puan (0-100): ");
if (scanf("%d", &puan) != 1) {          /* 1) Sayı mı? */
    printf("Hata: bir tamsayı girmelisiniz.\n");
    return 1;
}
if (puan < 0 || puan > 100) {           /* 2) Aralıkta mı? */
    printf("Hata: %d aralık dışında (0-100).\n", puan);
    return 1;
}
printf("Kabul edildi: %d\n", puan);      /* 3) Artık güvenle kullan */
```

GERÇEK ÇIKTI · İKİ GEÇERSİZ GİRDİ

```
$ ./dogrulama      # aralık dışı
Puan (0-100): 105
Hata: 105 aralık dışında (0-100).
[çıkış kodu: 1]

$ ./dogrulama      # sayı değil
Puan (0-100): abc
Hata: bir tamsayı girmelisiniz.
[çıkış kodu: 1]
```

- 1) Sayı mı? `scanf` 1 döndürmeli
- 2) Aralıkta mı? (De Morgan)
- Geçersizse ileti + `return 1`
- “Tekrar sor” bir **döngüdür** (H6)

03

Kaputun Altında

if → cmp + koşullu atlama · switch → atlama tablosu · dal tahmini

if (final_notu >= 50) → cmpl $49 + jle

H05_K10_kaput_if.c · satır 14–18 (C)

```
14      if (final_notu >= 50) {
15          printf("Baraj geçildi\n");
16      } else {
17          printf("FF\n");
18      }
```

gcc -O0 -S · main'den kesit

```
.L2:
    movl    -4(%rbp), %eax
    cmpl    $49, %eax
    jle     .L4
    leaq    .LC1(%rip), %rax
    movq    %rax, %rdi
    call    puts@PLT
    jmp     .L5
.L4:
    leaq    .LC2(%rip), %rax
    movq    %rax, %rdi
    call    puts@PLT
.L5:
```

- final_notu ile 49'u karşılaştır → bayraklar
- final_notu ≤ 49 ise else'e (.L4) ATLA ("≥ 50", derleyicide "≤ 49 değilse")
- printf("...\n") → puts (derleyici sadeleştirmesi)
- then bitti: else'i atla (.L5)

Aynı renk = aynı C satırının komutları (gcc -g'nin .loc yönergelerinden çıkarıldı). -O2'de de aynı kalıp: `cmpl $49, 12(%rsp)` ve `jle .L3`.

gcc -std=c17 -O0 -S -fno-asynchronous-unwind-tables -fno-stack-protector -fcf-protection=none -U_FORTIFY_SOURCE → H05_K10_kaput_if_O0.s

Yoğun case değerleri → atlama tablosu

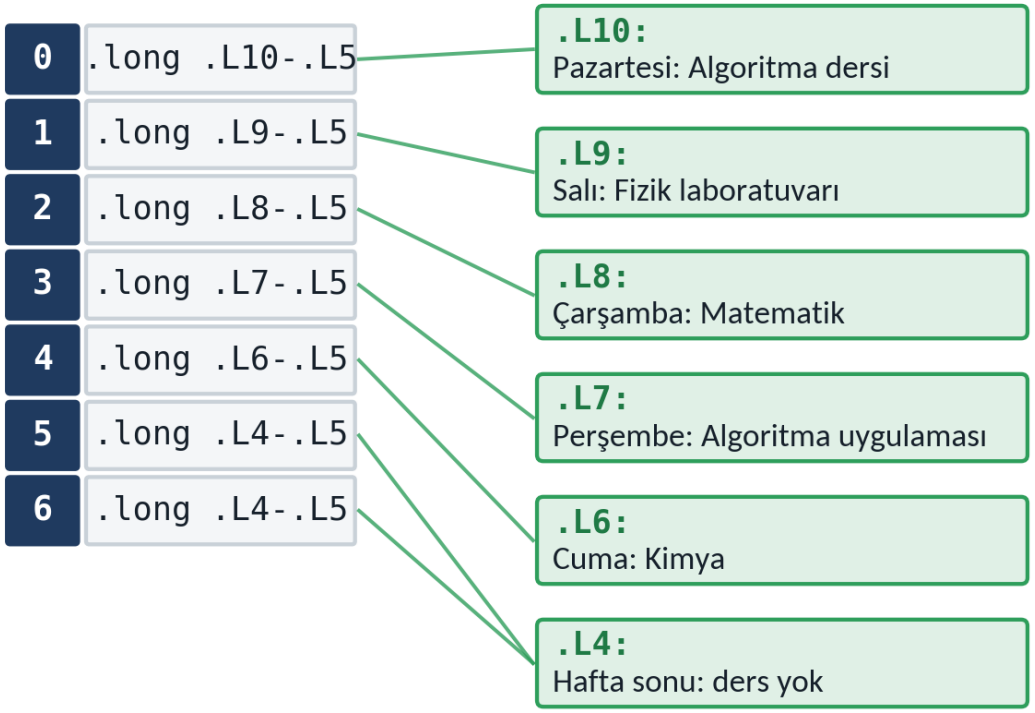
gcc -O2 · …_K07_…_O2.s

```
    movl    12(%rsp), %esi      gun değerini yazmaca al
    leal    -1(%rsi), %eax      indeks = gun − 1  (0…6)
    cmpl    $6, %eax            indeks > 6 mı? (işaretsiz!)
    ja      .L3                 evetse default'a (.L3)
    leaq    .L5(%rip), %rdx     tablonun adresi
    movslq  (%rdx,%rax,4), %rax tablodan 4 baytlık fark oku
    addq    %rdx, %rax          hedef = tablo + fark
    jmp     *%rax               HESAPLANAN adrese atla
```

Tek karşılaştırma iki sınırı birden denetler: gun ≤ 0 ise
gun − 1 işaretsiz yorumda çok büyük bir sayıdır → ja → default.

gun = 6 ve gun = 7 aynı hedefe (.L4) gider: art arda case etiketleri.
Kaç case olursa olsun: bir karşılaştırma + bir dolaylı atlama.

.L5: atlama tablosu (rodata)



H05_K07_switch_ders_programi.c · gcc -O2 -S (gerçek çıktı) · gün 6 ve 7 aynı hedefe (.L4) gider

Dal tahmini: işlemci neden “tahmin” eder?

İşlemci bir komut hattıdır: bir komut yürütülürken sonrakiler çoktan getirilmektedir.



Dalsız seçenek: ?: için GCC cmov (koşullu taşıma) üretebilir (K12) — atlama yoksa tahmin de gerekmez.

?: için GCC bu makinede -O0’da bile cmov üretti: cmovge %edx, %eax (H05_K12)

04

Örnek merdiveni

13 basamak: işaretten sınır değer testine

★ Örnek 1–3: işaret, tek/çift, iki sayının büyüğü

H05_01_isaret.c · satır 15–21

```c
15  if (sayi > 0) {
16      printf("%d pozitiftir.\n", sayi);
17  } else if (sayi < 0) {
18      printf("%d negatiftir.\n", sayi);
19  } else {
20      printf("Sayı sıfırdır.\n");
21  }
```

ÖRNEK 1 · ÜÇ DAL, ÜÇ TEST

```
Bir tamsayı girin: 7
7 pozitiftir.
Bir tamsayı girin: -3
-3 negatiftir.
Bir tamsayı girin: 0
Sayı sıfırdır.
```

H05_02_tek_cift.c · satır 15–19

```c
15  if (sayi % 2 == 0) {
16      printf("%d çifttir.\n", sayi);
17  } else {
18      printf("%d tektir.\n", sayi);
19  }
```

ÖRNEK 2 · −7

```
-7 tektir.
(-7 % 2 = -1)
```

−7 % 2 = **−1** (H4): `% 2 == 1` negatif tek sayıda yanlış olur.

H05_03_iki_sayi_buyuk.c · satır 26

```c
26  int buyuk2 = (a > b) ? a : b;
```

Örnek 3: H2 Örnek 3'ün if-else'i (satır 17–22) tek satırda `?:` ile.

★★ Örnek 4: üç sayının en büyüğü, üç çözüm

H05_04_uc_sayi_en_buyuk.c · satır 33–41

```c
/* Çözüm 2: && ile bileşik koşullar */
int eb2 = 0;
if (a >= b && a >= c) {
    eb2 = a;
} else if (b >= a && b >= c) {
    eb2 = b;
} else {
    eb2 = c;
}
```

Girdi	Beklenen	İç içe / && / ardışık	
4 9 7	9	9 / 9 / 9	✓
9 4 7	9	9 / 9 / 9	✓
4 7 9	9	9 / 9 / 9	✓
9 9 2	9	9 / 9 / 9	✓
5 5 5	5	5 / 5 / 5	✓
-3 -8 -1	-1	-1 / -1 / -1	✓

İç içe if: her zaman 2 karşılaştırma

&& ile: en kötü 4 karşılaştırma

Ardışık güncelleme: 2 karşılaştırma, N sayıya genişler (H6)

H2 Örnek 5'teki iki strateji + && ile yeni bir çözüm. Eşit ve negatif değerleri **mutlaka** test edin.

★★ Örnek 5: artık yıl, if zinciri ve tek ifade

H05_05_artik_yil.c · satır 17–29

```c
bool artik1 = false;
if (yil % 400 == 0) {
    artik1 = true;
} else if (yil % 100 == 0) {
    artik1 = false;
} else if (yil % 4 == 0) {
    artik1 = true;
} else {
    artik1 = false;
}

/* 2. yol: tek bileşik ifade (H4 Örnek 8) */
bool artik2 = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
```

ÇIKTI

```
$ ./artik_yil
Yıl girin: 1900
if zinciri : artık yıl değil
tek ifade  : artık yıl değil

$ ./artik_yil
Yıl girin: 2000
if zinciri : artık yıl
tek ifade  : artık yıl
```

- Zincir: **en özelden genele** (400 → 100 → 4)
- Tek ifade: H4 Örnek 8
- `bool`, `true`, `false` (stdbool.h)

★★ Örnek 6 (amiral): puandan harf notu

H05_06_harf_notu.c · satır 15–25

```c
15  if (puan < 0 || puan > 100) {                /* girdi doğrulama */
16      printf("Hata: %d geçersiz; puan 0-100 olmalı.\n", puan);
17      return 1;
18  }
19
20  printf("Harf notu: ");
21  if (puan >= 90) {
22      printf("AA\n");
23  } else if (puan >= 81) {
24      printf("BA\n");
25  } else if (puan >= 73) {
```

- Önce **girdi doğrulama**: `return 1`
- Sonra büyükten küçüğe merdiven (satır 21–39), RTEÜ tablosu
- Her basamak yalnızca alt sınırı sınar

ÇIKTI

```
$ ./harf_notu
Puanı girin (0-100): 81
Harf notu: BA

$ ./harf_notu
Puanı girin (0-100): 101
Hata: 101 geçersiz; puan 0-100 olmalı.
[çıkış kodu: 1]
```

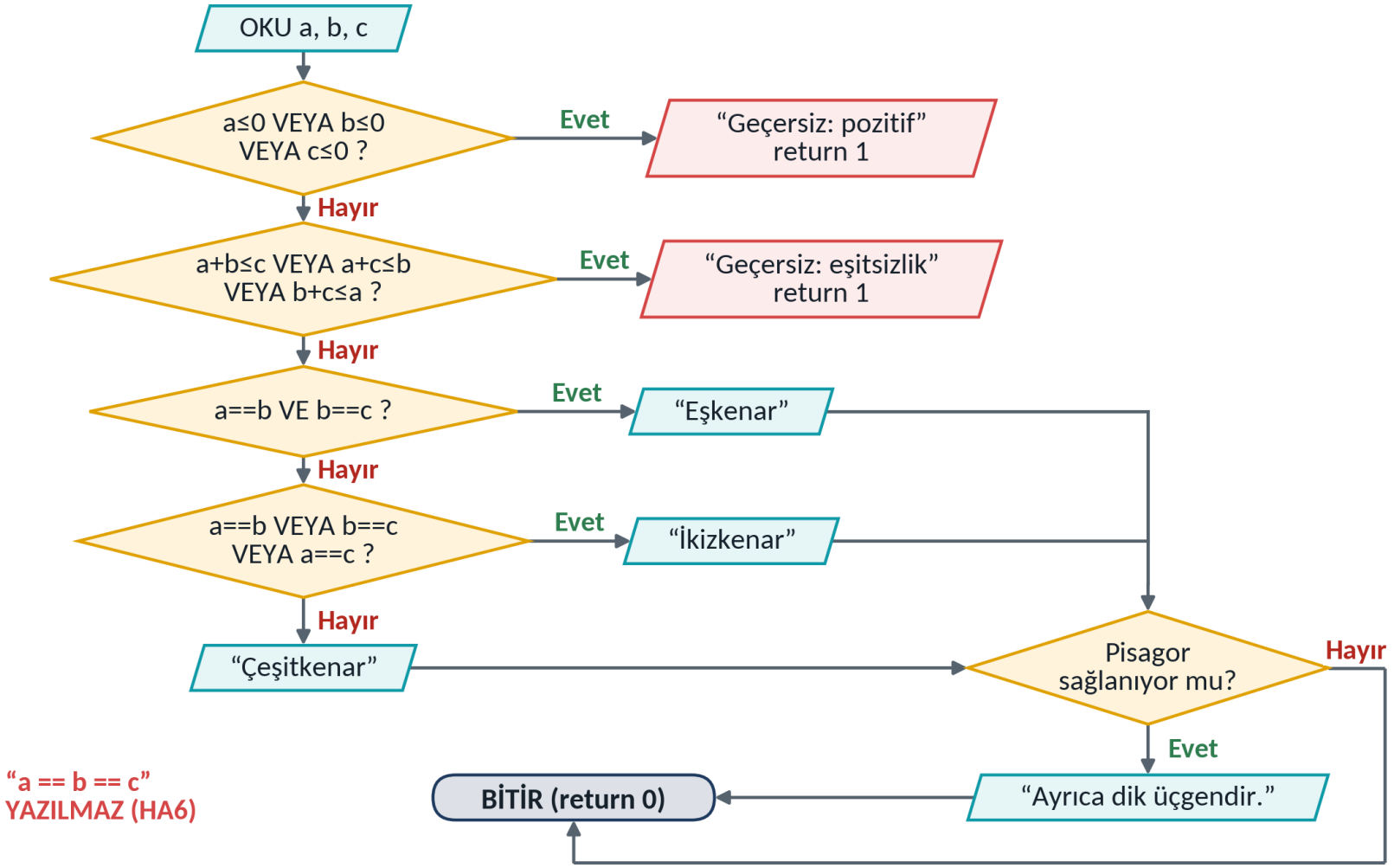
Örnek 6’nın izleme tablosu (girdi 81)

H05_06_harf_notu.c · satır 13–24

```
13  scanf("%d", &puan);
14
15  if (puan < 0 || puan > 100) {                /* girdi doğrulama */
16      printf("Hata: %d geçersiz; puan 0-100 olmalı.\n", puan);
17      return 1;
18  }
19
20  printf("Harf notu: ");
21  if (puan >= 90) {
22      printf("AA\n");
23  } else if (puan >= 81) {
24      printf("BA\n");
```

Adım	Satır	Koşul	Değer	Çıktı
1	13	–	–	
2	15	puan<0 || puan>100	0	
3	20	–	–	Harf notu:
4	21	puan >= 90	0	
5	**23**	**puan >= 81**	**1**	
6	24	–	–	BA

★★ Örnek 7: üçgen geçerliliği ve türü



Gerçek çalıştırmalar

- `3 4 5` → Çeşitkenar üçgen · Ayrıca dik üçgendir.
- `5 5 5` → Eşkenar üçgen
- `5 5 8` → İkizkenar üçgen
- `4 6 7` → Çeşitkenar üçgen
- `5 13 12` → Çeşitkenar üçgen · Ayrıca dik üçgendir.
- `1 2 3` → Geçersiz: üçgen eşitsizliği sağlanmıyor.
- `0 4 5` → Geçersiz: kenarlar pozitif olmalı.

★★ Örnek 8: ikinci derece denklem, Δ'nın üç dalı

H05_08_ikinci_derece.c · satır 26–38

```c
if (delta > 0) {
    double x1 = (-b + sqrt(delta)) / (2.0 * a);
    double x2 = (-b - sqrt(delta)) / (2.0 * a);
    printf("İki farklı gerçel kök: x1 = %.4f, x2 = %.4f\n", x1, x2);
} else if (delta == 0) {
    double x = -b / (2.0 * a);
    printf("Çakışık (çift) kök: x = %.4f\n", x);
} else {
    double gercel = -b / (2.0 * a);
    double sanal = sqrt(-delta) / fabs(2.0 * a);
    printf("Gerçel kök yok; karmaşık kökler: %.4f ± %.4fi\n",
           gercel, sanal);
}
```

GERÇEK ÇIKTILAR · 1 -3 2 · 1 2 1 · 1 0 4

```
Delta = 1
İki farklı gerçel kök: x1 = 2.0000, x2 = 1.0000
Delta = 0
Çakışık (çift) kök: x = -1.0000
Delta = -16
Gerçel kök yok; karmaşık kökler: 0.0000 ± 2.0000i
```

- `delta` **tamsayı**: `== 0` güvenli (bkz. HA8)
- `sqrt`, `fabs`: `math.h`, derlemede `-lm`
- H4 Örnek 11'deki köprü: Δ < 0 artık bir dal

★★ Örnek 9: dört işlem hesap makinesi (switch + if)

H05_09_hesap_makinesi.c · satır 27–37

```c
case '/':
    if (b == 0) {                       /* sıfıra bölme denetimi */
        printf("Hata: sıfıra bölme tanımsızdır.\n");
    } else {
        printf("%d / %d = %.2f\n", a, b, (double)a / b);
    }
    break;
default:
    printf("Bilinmeyen işlem: '%c'\n", islem);
    break;
}
```

ÇIKTI

```
$ ./hesap
İşlem girin (ör. 12 / 4): 12 / 4
12 / 4 = 3.00

$ ./hesap
İşlem girin (ör. 12 / 4): 5 / 0
Hata: sıfıra bölme tanımsızdır.

$ ./hesap
İşlem girin (ör. 12 / 4): 8 ^ 2
Bilinmeyen işlem: '^'
```

- `case '+', '-', '*'`: satır 18–26
- `case '/'` içinde **if**: sıfıra bölme
- `default`: beklenmeyen işlem

★★ Örnek 10: ayın gün sayısı, art arda case etiketleri

H05_10_ay_gun.c · satır 17–26

```c
switch (ay) {
case 1:
case 3:
case 5:
case 7:
case 8:
case 10:
case 12:
    gun = 31;          /* yedi etiket, tek gövde */
    break;
```

1 Ocak 31	2 Şubat 28/29	3 Mart 31	4 Nisan 30	5 Mayıs 31	6 Haziran 30
7 Temmuz 31	8 Ağustos 31	9 Eylül 30	10 Ekim 31	11 Kasım 30	12 Aralık 31

■ 31 gün ■ 30 gün ■ Şubat: case içinde if-else

GERÇEK ÇALIŞTIRMALAR (AY YIL → SONUÇ)

```
2 2024  → 2. ay (2024) 29 gün çeker.
2 2026  → 2. ay (2026) 28 gün çeker.
4 2026  → 4. ay (2026) 30 gün çeker.
12 2026 → 12. ay (2026) 31 gün çeker.
2 1900  → 2. ay (1900) 28 gün çeker.
13 2026 → Hata: 13 geçerli bir ay değil (1-12).
          [çıkış kodu: 1]
```

ÖRNEK MERDİVENİ ★★ · MÜHENDİS GÖZÜYLE

★★ Örnek 11: PIN doğrulama ve kullanıcı yetkisi

H05_11_pin_yetki.c · satır 25–31

```
25  bool ogrenci_mi = (kullanici == OGRENCI_NO && pin == OGRENCI_PIN);
26  bool hoca_mi = (kullanici == HOCA_NO && pin == HOCA_PIN);
27
28  if (!ogrenci_mi && !hoca_mi) {   /* De Morgan: !(ogrenci_mi || hoca_mi) */
29      printf("Erişim reddedildi: kullanıcı no veya PIN hatalı.\n");
30      return 1;
31  }
```

- `&&`: numara ile PIN **aynı hesaba** ait olmalı
- Ret koşulu De Morgan: `!ogrenci_mi && !hoca_mi`
- İleti **hangi bilginin** yanlış olduğunu söylemez
- **Yalnızca eğitsel:** gerçek sistemde PIN koda yazılmaz; **özeti (hash)** saklanır; **deneme sayısı** sınırlanır (H6)

GERÇEK ÇALIŞTIRMA · BAŞKASININ PIN'İ

```
$ ./pin      # başkasının PIN'i
Kullanıcı no: 1001
PIN         : 7391
Erişim reddedildi: kullanıcı no veya PIN hatalı.
[çıkış kodu: 1]
```

★★★ Örnek 12 (amiral): kademeli elektrik faturası

H05_12_elektrik_faturasi.c · satır 27–37

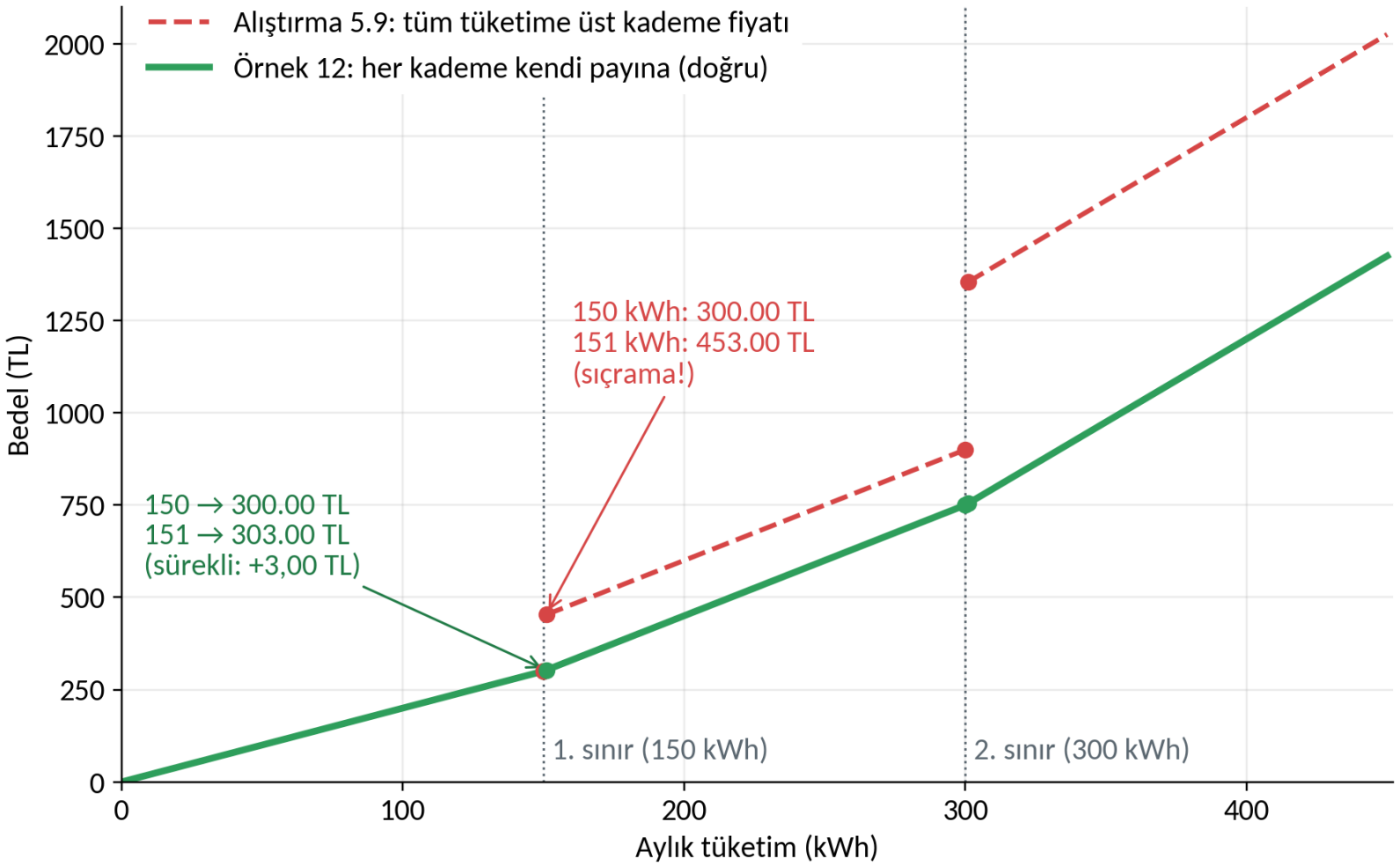
```c
27  if (tuketim <= KADEME1_UST) {
28      kademe = 1;
29      bedel = tuketim * FIYAT1;
30  } else if (tuketim <= KADEME2_UST) {
31      kademe = 2;
32      bedel = KADEME1_UST * FIYAT1 + (tuketim - KADEME1_UST) * FIYAT2;
33  } else {
34      kademe = 3;
35      bedel = KADEME1_UST * FIYAT1 + (KADEME2_UST - KADEME1_UST) * FIYAT2
36              + (tuketim - KADEME2_UST) * FIYAT3;
37  }
```

- İlk 150 kWh 2,00 · 151–300 3,00 · üstü 4,50 TL (örnek)
- Üst fiyat **yalnızca o kademeye düşen kısma**
- Sınır testleri: 150/151 · 300/301
- Sabitler: tarife değişince yalnızca onlar değişir

GERÇEK ÇIKTI · SINIR TESTLERİ

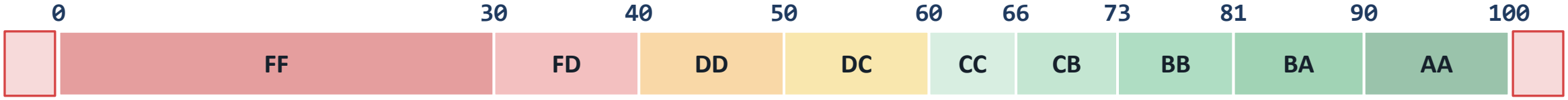
```
Tüketim: 150 kWh | Kademe: 1 | Bedel: 300.00 TL
Tüketim: 151 kWh | Kademe: 2 | Bedel: 303.00 TL
Tüketim: 300 kWh | Kademe: 2 | Bedel: 750.00 TL
Tüketim: 301 kWh | Kademe: 3 | Bedel: 754.50 TL
```

Sınır testi bir “sıçramayı” yakalar



- Yeşil: Örnek 12 (doğru)
- Kırmızı: Alıştırma 5.9, **tüm tüketime** üst fiyat
- 150 → 151 kWh: 300,00 → **453,00 TL**
- Aralık ortasından test (200) bunu **yakalayamaz**

★★★ Örnek 13: sınır değer test tablosu (30 test)



–1: geçersiz *Her sınır için üç test: bir altı · kendisi · bir üstü (ekranda sınırın iki yanı)* **101: geçersiz**

Girdi	Beklenen (elle)	Gerçek çıktı	
-1	GEÇERSİZ	GEÇERSİZ	✓
0	FF başarısız	FF başarısız	✓
29	FF başarısız	FF başarısız	✓
30	FD başarısız	FD başarısız	✓
39	FD başarısız	FD başarısız	✓
40	DD başarısız	DD başarısız	✓
49	DD başarısız	DD başarısız	✓
50	DC şartlı başarılı	DC şartlı başarılı	✓
59	DC şartlı başarılı	DC şartlı başarılı	✓
60	CC başarılı	CC başarılı	✓

Girdi	Beklenen (elle)	Gerçek çıktı	
65	CC başarılı	CC başarılı	✓
66	CB başarılı	CB başarılı	✓
72	CB başarılı	CB başarılı	✓
73	BB başarılı	BB başarılı	✓
80	BB başarılı	BB başarılı	✓
81	BA başarılı	BA başarılı	✓
89	BA başarılı	BA başarılı	✓
90	AA başarılı	AA başarılı	✓
100	AA başarılı	AA başarılı	✓
101	GEÇERSİZ	GEÇERSİZ	✓

Betik çıktısı: **Toplam: 30 test, geçen: 30, kalan: 0** · 30 testin tamamı: kodlar/H05_13_sinir_testi.sh ve ders notu

Hata Avcısı: atama mı, karşılaştırma mı? Zincir karşılaştırma

HA1 · if (x = 5)

hata_avcisi/H05_HA1_atama_kosul.c · satır 11

```
11  if (kalan_hak = 3) {              /* niyet: kalan_hak == 3 mü? */
```

GCC 13 (-WALL)

```
warning: suggest parentheses around assignment used as truth value [-Wparentheses]
```

Atama yapıldı, değer 3 → her zaman doğru. Gerçek çıktı: “Tam hakkınız var: 3” (hak 0 iken!)

Çözüm: == kullanın.

HA6 · 0 < x < 10

hata_avcisi/H05_HA6_zincir_karsilastirma.c · satır 11

```
11  if (0 < x < 10) {                 /* matematikte: 0 < x ve x < 10 */
```

GCC 13 (-WALL)

```
warning: comparison of constant '10' with boolean expression is always true [-Wbool-compare]
warning: comparisons like 'X<=Y<=Z' do not have their mathematical meaning [-Wparentheses]
```

`0 < x` 0/1 üretir; 0/1 her zaman < 10. Gerçek çıktı (x = 25): “25, (0, 10) aralığında.”

Çözüm: `0 < x && x < 10`

Hata Avcısı: uyarı var mı, hangi seçenekle?

Hata	GCC 13 iletisi (özet)	Seçenek	Gerçek sonuç
HA2 if (puan >= 60);	suggest braces around empty body in an 'if' statement	-Wextra (-Wempty-body); -Wall da -Wmisleading-indentation ile yakaladı	puan 40: “Tebrikler, geçtiniz!”
HA3 parantezsiz gövde	this 'if' clause does not guard...	-Wall (-Wmisleading-indentation)	final 70: “Uyarı sayısı: 1”
HA5 sallanan else	suggest explicit braces to avoid ambiguous 'else'	-Wall (-Wdangling-else)	yas 16: yalnızca “Denetim bitti.”
HA7 break unutma	this statement may fall through	yalnızca -Wextra (-Wimplicit-fallthrough)	seçim 1: “Not SİLME açıldı!” da
HA8 toplam == 0.3	(uyarı yok)	yalnızca -Wfloat-equal	“Eşit değil! toplam - 0.3 = 5.551e-17”

MÜHENDİS GÖZÜYLE

goto fail'den dört mühendislik dersi

hata_avcisi/H05_HA4_goto_fail_yeniden_duzeltilmis.c · satır 14–21

```
14  if ((hata = ozet_hatasi) != 0) {
15      goto fail;
16      goto fail;              /* aynı kopyalama hatası artık zararsız */
17  }
18
19  printf("2. adım: sunucunun imzası doğrulanıyor\n");
20  if (imza_gecerli == 0) {
21      hata = -1;
```

DÜZELTİLMİŞ SÜRÜM · GERÇEK ÇIKTI

```
1. adım: el sıkışma verisinin özeti hesaplanıyor
2. adım: sunucunun imzası doğrulanıyor
Sonuç: doğrulama BAŞARISIZ, bağlantı reddedildi
```

- **Süslü parantez her zaman:** yinelenen satır blokta kalır, zararsızdır
- **Uyarıları hata say:** `-Werror`
- **Olumsuz testi yaz:** "sahte imza reddediliyor mu?"
- **Erişilemez kod** bir alarmdır: inceleme ve statik çözümleme
- **Girdiye güvenme:** dışarıdan gelen her değer denetlenir

MİNİ-OBS PROJESİ

Mini-OBS v0.5: doğrulama, harf notu, durum

- v0.4 kuralları aynen: `0.40*vize + 0.60*final`, `(int)(ortalama + 0.5)`
- Vize/final **0–100 dışı** → ileti + `return 1`
- Final barajı: `final < 50` → **FF**, "BAŞARISIZ (final barajı)"
- Harf: RTEÜ tablosu, `yuvarlanmis` üzerinden else-if
- Durum: ≥ 60 **BAŞARILI** · 50–59 **ŞARTLI** · < 50 **BAŞARISIZ**

GERÇEK ÇIKTI · T3 (VİZE 69, FİNAL 53)

```
Öğrenci no   : 2301010045
Vize  (%40) :  69.00
Final (%60) :  53.00
Ortalama     :  59.40
Yuvarlanmış :  59
Harf notu    : DC
Durum        : ŞARTLI BAŞARILI (YANO ≥ 2,00 ise)
```

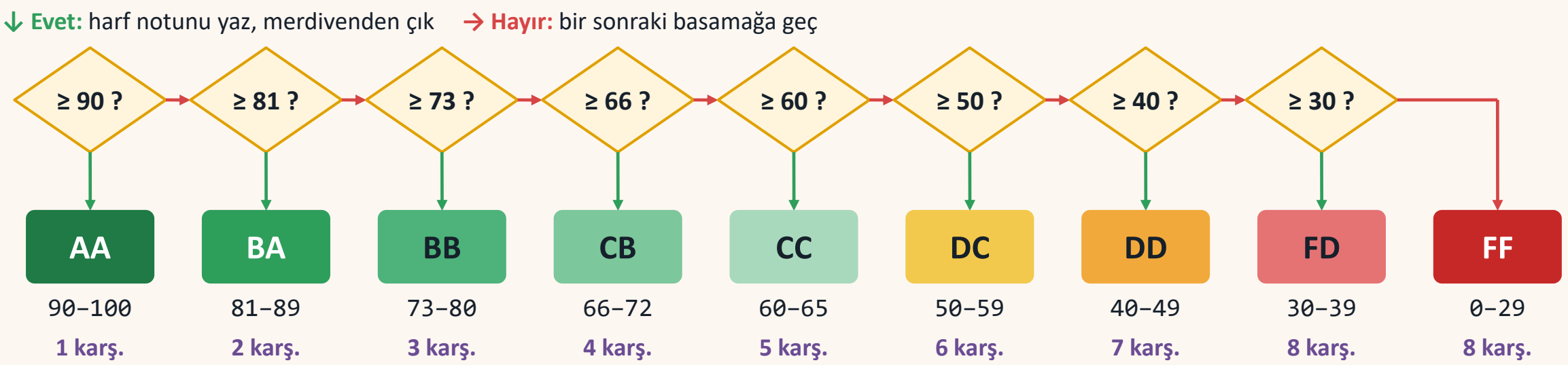
	Vize / Final	Ort. → Yuv.	Harf	Durum (gerçek)	
T1	92 / 95	93.80 → 94	AA	BAŞARILI	✓
T2	70 / 52.5	59.50 → 60	CC	BAŞARILI	✓
T3	69 / 53	59.40 → 59	DC	ŞARTLI BAŞARILI*	✓
T4	100 / 45	67.00 → 67	FF	BAŞARISIZ (final barajı)	✓
T5	101	—	—	HATA (vize), çıkış 1	✓
T6	80 / -5	—	—	HATA (final), çıkış 1	✓
T7	80 / 50	62.00 → 62	CC	BAŞARILI	✓
T8	80 / 49	61.40 → 61	FF	BAŞARISIZ (final barajı)	✓
T9	0 / 100	60.00 → 60	CC	BAŞARILI	✓
T10	30 / 60	48.00 → 48	DD	BAŞARISIZ	✓
T11	80 / 81	80.60 → 81	BA	BAŞARILI	✓

** YANO ≥ 2,00 ise · beklenenler elle hesaplandı, 11/11 uyuştu (91_Proje_MiniOBS/v0.5)*

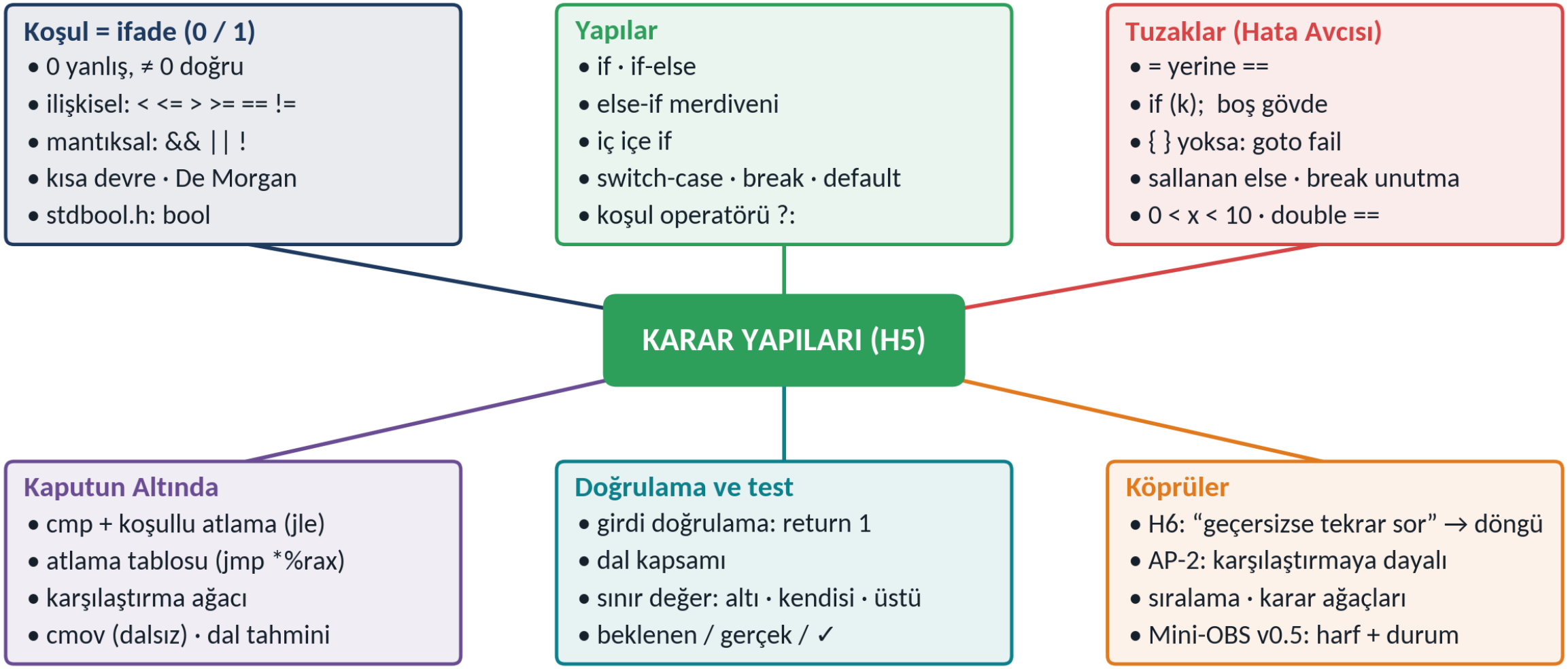
AP-2 KÖPRÜSÜ

Karşılaştırma: sıralamanın temel işlemi

- Karşılaştırmaya dayalı sıralamanın maliyeti **karşılaştırma sayısıyla** ölçülür (Örnek 4'te saydık)
- Harf merdiveninde AA için **1**, FD ve FF için **8** karşılaştırma; ortadan bölen dengeli bir **karar ağacı** 9 sonucu en fazla **4** karşılaştırmayla ayırır
- Seyrek switch'in karşılaştırma ağacı aynı fikir: **ikili arama** sezgisi (biçimsel analiz AP-2'de)



Özet: kavram haritası



KAPANIŞ

Büyük Sorunun yanıtı ve H6'ya köprü

Neden?

- Fazladan `goto fail;`, parantezsiz `if`'ten sonra **koşulsuz** çalıştı
- İmza doğrulama **erişilemez** oldu; hata değişkeni 0 → "başarılı"

Nasıl önlenirdi?

- `{ }` her zaman
- `-Wall -Wextra` ve `-Werror`
- GCC: -Wmisleading-indentation
- Olumsuz test + kod incelemesi

Sıradaki: H6 Döngüler

- "Geçersizse **tekrar sor**" = döngü
- `while`, `do-while`, `for`; bu haftanın koşulları döngü koşulu olur
- Mini-OBS v0.6: N öğrenci

Girinti insan içindir; derleyici için anlam süslü parantezdedir.

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `!(t < 0 || t > 40)` koşulunu `!` kullanmadan yazın. Hangi kuralı kullandınız?

2 RTEÜ harf notu tablosunda BB/BA sınırı için hangi üç testi seçersiniz? Beklenen çıktıları yazın.

3 Bir `switch`'te `case 1:` gövdesinde `break` yoksa 1 girildiğinde ne olur? GCC hangi seçenekle uyarır?

KAPANIŞ

Haftaya hazırlık ve kaynaklar

- **H6 · Döngüler I:** `while`, `do-while`, `for`; sayaç, toplam, nöbetçi değer, girdi doğrulama döngüsü.
  - Mini-OBS v0.6: “geçersizse tekrar sor” ve N öğrenci
- **Atölye:** harf notu + üçgen + hesap makinesi üçlüsü; Örnek 12’ye dördüncü kademe için **önce test tablosu**.
- **Mini-OBS v0.5:** 11 kabul testi (ders notu Bölüm 7; referans çıktılar 91_Proje_MiniOBS/v0.5).
- **Web crash course:** karar ağacı, “Hangi dal çalışır?” kartları, C ↔ assembly görünümü, De Morgan oynatıcısı; 10 soruluk quiz.
- **Okuma:** NIST (2014) CVE-2014-1266; Langley (2014) “Apple’s SSL/TLS bug”; Bryant & O’Hallaron (2016), makine düzeyinde denetim akışı.